

Relatório de Auditoria de Segurança — GraoSys

Relatório técnico de auditoria de segurança de aplicação

Data: 15/09/2026

Escopo auditado: graosys-backend/ (Express + TypeORM + PostgreSQL) e graosys-frontend/ (React + Vite), incluindo todos os controllers, middlewares, rotas, entidades, configuração de banco, arquivos .env/.env.example, vercel.json e histórico completo do repositório Git (3 commits: 788fec1, 50a8364, d3232bd, 58ac83a).

Nota metodológica

Stack detectada: backend Node.js/Express + TypeScript, ORM TypeORM sobre PostgreSQL, autenticação via JWT stateless (sem sessão em banco), multi-tenancy implementado por coluna `tenant\_id` manual em cada tabela (não é Supabase/RLS). Frontend: React 18 + Vite + TypeScript + React Router + Axios, sem SSR. Deploy via Vercel (`vercel.json` de rewrites em backend e frontend), sem Docker/Kubernetes/Terraform no repositório.

Mapeamento das 5 categorias para esta stack: 1. Isolamento de tenant → não há RLS (Postgres puro via TypeORM); o mecanismo é o filtro manual `tenant\_id: req.user.tenant\_id` que cada controller deve aplicar em toda query. 2. Permissão no navegador → comparação entre os gates de `role`/`permissions` no React Router (`AdminRoute`, tela de Controle de Acesso) e o middleware `requireRole`/checagem de `permissions` no Express. 3. IDOR → percorridos sistematicamente todos os handlers de `SessionController`, `TenantController`, `UserController`, `ClientController`, `GrainContractController`, `BillingController`, `ProductController`, `ProductTableController`, `DashboardController`, `EmailController` (rotas em `src/app/routes/index.ts`). 4. Chaves expostas → variáveis de ambiente e seus fallbacks em código (`authMiddleware.ts`, `SessionController.ts`, `data-source.ts`), `.env`/`.env.example`, `vercel.json`, e histórico completo do `git log --all -p`. 5. XSS → grep exaustivo por `dangerouslySetInnerHTML`/`innerHTML`/`eval`/`new Function` em todo o frontend React, e revisão de todo ponto do backend que gera HTML (`EmailController`, templates de e-mail via `nodemailer`).

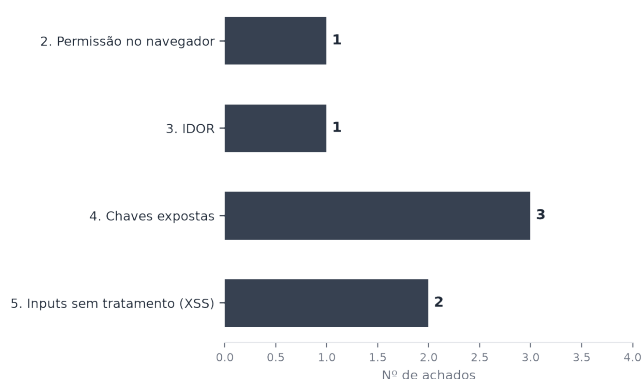
Resumo Executivo

Foram identificados **7 achados** nas cinco categorias avaliadas. Dois achados foram classificados como **Críticos** (quebra de isolamento entre tenants e ausência total de enforcement de permissões). A aplicação demonstrou, por outro lado, uma base sólida de isolamento de tenant nas operações de leitura e ausência de vetores clássicos de SQL Injection e XSS no frontend.

Status: todos os 7 achados foram corrigidos em 15/09/2026 — ver detalhe da correção aplicada em cada achado e no changelog de código (graosys-backend). Este relatório foi atualizado após a remediação para manter o histórico da auditoria original.



Crítica (2) Alta (3) Média (1) Baixa (1)



Severidade	Qtde
Crítica	2
Alta	3
Média	1
Baixa	1
Total	7

Pontos Fortes (verificados no código)

Itens abaixo foram ativamente verificados no código-fonte e representam controles corretos, não apenas ausência de achados. Servem como evidência de cobertura da auditoria.

■ Isolamento de tenant nas leituras (categoria 1)

Todos os métodos de listagem/busca/agregação/relatório auditados — `ClientController` (`getAll`, `getById`, `getCnpjCpf`), `GrainContractController` (`getAll`, `getById`, `getReport`), `BillingController` (`getAll`, `getById`, `getByNumberContract`, `getSummary`), `ProductController/ProductTableController` (`getAll`, `getById`) e `DashboardController.getSummary` — filtram corretamente por ``tenant_id``: `req.user.tenant_id`, valor extraído do JWT assinado pelo servidor, nunca de input do cliente. Nenhum handler de listagem aceita um ``tenant_id`` vindo de `query/body/params`.

■ Gate de papel (admin) aplicado no backend onde existe equivalente no frontend

Todas as rotas administrativas expostas no frontend (rotas ``/admin/*``, gated por ``AdminRoute`` em `Routes.tsx`) têm o equivalente ``requireRole("admin")`` no backend: gestão de usuários (`GET/POST/PATCH/DELETE /api/users`), escrita de produtos e mesas de produtos, atualização de dados do tenant e delete de clientes/contratos/cobranças. Não foi encontrada nenhuma ação administrativa exposta na UI que fique apenas com o gate visual do frontend sem o correspondente no servidor.

■ Autenticação e senha tratadas corretamente

`SessionController.login` usa `bcrypt.compare`, nunca retorna o hash da senha, bloqueia login de tenants com ``status === "suspended"`` e de usuários com ``active === false``. `resetPassword` exige a senha atual antes de trocar. `UserController` nunca devolve o campo ``password`` em nenhum dos 4 endpoints que retornam dados de usuário (select explícito de colunas ou desestruturação).

■ Sem SQL Injection

As duas únicas construções de query manual do projeto (``createQueryBuilder`` em `GrainContractController.getReport` e `BillingController.getSummary`) usam exclusivamente parâmetros nomeados (``:tenant_id``, ``:year``, ``:crop``, etc.) via ``andWhere("c.crop = :crop", { crop })``, sem nenhuma concatenação de string do usuário na query.

■ Sem XSS no frontend

Busca exaustiva por ``dangerouslySetInnerHTML``, ``innerHTML``, ``v-html``, ``eval`` e ``new Function`` em todo o diretório ``graosys-frontend/src`` não retornou nenhuma ocorrência. Toda a renderização de dados do usuário passa pelo escaping automático do React (JSX).

■ Segredos não versionados no Git

``.env`` está listado em ``.gitignore`` desde o commit inicial e uma varredura em ``git log --all -p`` não encontrou nenhum valor de segredo real (não-placeholder) commitado — apenas os defaults de exemplo já reportados como achados (F3/F5).

Pontos Fracos (riscos centrais)

O risco central da aplicação está na camada de **escrita**: enquanto as leituras respeitam o isolamento por tenant, os métodos de criação/atualização aceitam campos não-whitelist do corpo da requisição (mass assignment), permitindo reescrever registros de qualquer tenant (F1). Some-se a isso um sistema de permissões granulares que existe apenas na interface administrativa, sem qualquer enforcement no backend (F2) — as duas falhas críticas do relatório. Defaults inseguros de segredos (F3, F5) e ausência de sanitização de HTML em e-mails (F4, F6) completam o quadro de riscos que dependem de configuração de deploy ou de conteúdo malicioso inserido por um usuário autenticado.

Achados Detalhados

Nota de cobertura — Categoria 1 (isolamento de tenant): não há achado autônomo nesta categoria. Todas as operações de leitura (listagem, busca, agregação, relatório) auditadas filtram corretamente por `tenant\_id` (ver seção "Pontos Fortes"). O único ponto em que o isolamento de tenant é efetivamente quebrado ocorre na camada de **escrita** (create/update), via mass assignment de `id`/`tenant\_id` — reportado como **F1**, classificado sob a categoria 3 (IDOR) por sua causa raiz técnica.

Tabela-resumo

Sev.	Arquivo:linha	Descrição	Status
Crítica	ClientController.ts:9, 50	[F1] Mass assignment de chave primária permite sequestro de registros de qualquer tenant	Corrigido
Crítica	index.ts:44-92	[F2] Sistema de permissões granulares (view/create/edit/delete por módulo) nunca é verificado no backend	Corrigido
Alta	authMiddleware.ts:32	[F3] JWT_SECRET com fallback hardcoded "secret" sem validação de startup	Corrigido
Alta	EmailController.ts:88-109	[F4] Endpoint de e-mail customizado aceita HTML arbitrário sem sanitização e sem restrição de papel	Corrigido
Alta	data-source.ts:39-41	[F5] Credencial de banco de dados com default hardcoded "postgres/postgres"	Corrigido
Média	EmailController.ts:112-149	[F6] HTML de e-mail transacional de contrato interpola campos do usuário sem escape	Corrigido
Baixa	User.ts:24	[F7] Coluna password sem select:false — depende de disciplina manual em cada query	Corrigido

Detalhamento por achado

Crítica

[F1] Mass assignment de chave primária permite sequestro de registros de qualquer tenant

Categoria: 3. IDOR**Arquivo(s):linha(s):** graosys-backend/src/app/controllers/ClientController.ts:9, 50 |

graosys-backend/src/app/controllers/GrainContractController.ts:9-16, 51 |

graosys-backend/src/app/controllers/BillingController.ts:8, 48 | graosys-backend/src/app/controllers/ProductController.ts:11, 33, 50, 72

```
// ClientController.update (linha 46-53)
const client = await clientRepo.findOne({ where: { id: req.params.id, tenant_id: req.user.tenant_id }
});
if (!client) return res.status(404)...;
Object.assign(client, req.body); // <-- body pode conter "id" e "tenant_id"
await clientRepo.save(client);

// ClientController.create (linha 8-11) - mesmo padrão no POST
const client = clientRepo.create({ ...req.body, tenant_id: req.user.tenant_id });
await clientRepo.save(client);
```

Por que é explorável: O TypeORM decide INSERT vs UPDATE em tempo de save() consultando o banco pelo valor ATUAL da chave primária do objeto (SubjectDatabaseEntityLoader.load(), que gera "WHERE id IN (...)\" usando entity.id no momento do save — confirmado lendo node_modules/typeorm/persistence/SubjectDatabaseEntityLoader.js e Subject.js:230). Como o `id` é uma coluna comum (@PrimaryColumn, UUID setado no client) e tanto o create() (spread de `...req.body`) quanto o update() (Object.assign(entity, req.body)) aceitam QUALQUER campo do body sem whitelist, um atacante autenticado pode enviar POST /api/clients (ou PATCH /api/clients/:id) com `{ "id": "<uuid-de-outro-tenant>", ... }`. A checagem de posse (`tenant\_id: req.user.tenant\_id`) só é aplicada no SELECT inicial, que localiza um registro do PRÓPRIO atacante — mas o `save()` final grava por PK, ignorando esse filtro, sobrescrevendo o registro-alvo com os dados do atacante e o `tenant\_id` do atacante. O mesmo padrão idêntico se repete em GrainContractController, BillingController, ProductController e ProductTableController (create e update).

Condição de explorabilidade: Pré-condição: o atacante precisa conhecer/adivinhar o UUID do registro-alvo (não é sequencial, mas UUIDs circulam em URLs da própria aplicação, relatórios exportados, logs, e-mails de contrato — qualquer usuário autenticado, mesmo com plano trial, pode disparar o ataque).

Impacto: Reescrita/hijack de registros (clientes, contratos, cobranças, produtos, mesas de preço) de QUALQUER outro tenant do SaaS — quebra total do isolamento multi-tenant na camada de escrita.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: criado utilitário `pickFields()` (graosys-backend/src/utils/pickFields.ts) e aplicado em create/update de ClientController, GrainContractController, BillingController, ProductController e ProductTableController — `id` e `tenant\_id` nunca mais são aceitos a partir do body.

Crítica**[F2] Sistema de permissões granulares (view/create/edit/delete por módulo) nunca é verificado no backend****Categoria:** 2. Permissão no navegador**Arquivo(s):linha(s):** graosys-backend/src/app/routes/index.ts:44-92 |

graosys-backend/src/app/controllers/UserController.ts:8, 16, 55, 60 |

graosys-frontend/src/pages/Admin/AccessControl/index.tsx:1-121

```
// routes/index.ts – únicas verificações de papel existentes:
router.get("/api/clients", client.getAll); // sem check de permissão
router.post("/api/clients", client.create); // sem check de permissão
router.patch("/api/clients/:id", client.update); // sem check de permissão
router.post("/api/contracts", contract.create); // sem check de permissão
router.post("/api/billings", billing.create); // sem check de permissão
// requireRole("admin") só existe em rotas de admin/produtos/usuários e nos deletes
```

Por que é explorável: A tela /admin/access (AdminAccessControlPage) permite ao admin configurar, por usuário não-admin, quais ações ("view", "create", "edit", "delete") ele pode executar em cada módulo (contracts, clients, execution, billing, reports) e persiste isso em `User.permissions` via PATCH /api/users/:id. Buscando `permissions` em todo o backend (grep), o único uso é: gravar o campo, devolvê-lo no login/perfil e no PATCH de usuário — NENHUM controller (Client, GrainContract, Billing, Product) lê ou valida `req.user.permissions` antes de autorizar create/edit. O frontend também não usa `permissions` em nenhuma página (nenhuma ocorrência de `hasPermission`/checagem condicional fora do AccessControl). Ou seja: a feature de permissões granulares é inteiramente decorativa — dá ao administrador uma falsa sensação de controle de acesso por módulo/ação, quando na prática QUALQUER usuário autenticado (independente da configuração) tem acesso total de criação/edição a contratos, clientes, cobranças e relatórios, só ficando de fora as ações de delete e as telas administrativas (que dependem apenas de `role === "admin"`, não de `permissions`).

Condição de explorabilidade: Basta ser um usuário autenticado com role "user" — nenhuma configuração especial necessária; o gate simplesmente não existe.

Impacto: Falha de segregação de funções: um usuário configurado para "somente visualizar contratos" pode, na prática, criar/editar/enviar e-mails de contratos, cobranças e clientes livremente.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: adicionado middleware `requirePermission(module, action)` (authMiddleware.ts) e aplicado em todas as rotas de view/create/edit de contracts, clients, billing, reports e execution (routes/index.ts). Admins continuam com acesso total; usuários não-admin agora dependem da configuração feita em /admin/access.

Alta**[F3] JWT_SECRET com fallback hardcoded "secret" sem validação de startup****Categoria:** 4. Chaves expostas**Arquivo(s):linha(s):** graosys-backend/src/app/middlewares/authMiddleware.ts:32 |
graosys-backend/src/app/controllers/SessionController.ts:45

```
// authMiddleware.ts:32
const decoded = jwt.verify(token, process.env.JWT_SECRET || "secret") as TokenPayload;

// SessionController.ts:45 (assinatura do token no login)
const token = jwt.sign({ ... }, process.env.JWT_SECRET || "secret", { expiresIn: "8h" });
```

Por que é explorável: Se a variável de ambiente `JWT\_SECRET` não estiver definida no ambiente de execução (erro de configuração no deploy, container mal configurado, etc.), a aplicação assina e valida tokens usando a string literal `"secret"` — um segredo público, presente no código-fonte. Não existe nenhuma checagem de startup (em server.ts/api/index.ts) que aborte a inicialização caso `JWT\_SECRET` esteja ausente ou seja igual ao valor default do `.env.example` (`your-super-secret-key-change-in-production`).

Condição de explorabilidade: Explorável apenas se a variável de ambiente estiver ausente em produção (falha de configuração de deploy) — nesse cenário, qualquer pessoa pode forjar um JWT válido (inclusive `role: "admin"` e `tenant\_id` arbitrário) usando a chave pública "secret", obtendo acesso total a qualquer tenant sem autenticação.

Impacto: Personificação total de qualquer usuário/tenant caso o segredo não seja configurado corretamente no ambiente de deploy.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: criado `validateEnv()` (src/config/validateEnv.ts), chamado no bootstrap (server.ts e api/index.ts), que aborta o startup em produção se `JWT\_SECRET` estiver ausente, curto ou igual a um valor padrão conhecido. O fallback hardcoded foi isolado em `src/config/jwtSecret.ts`, usado apenas fora de produção.

Alta**[F4] Endpoint de e-mail customizado aceita HTML arbitrário sem sanitização e sem restrição de papel****Categoria:** 5. Inputs sem tratamento (XSS)**Arquivo(s):linha(s):** graosys-backend/src/app/controllers/EmailController.ts:88-109 | graosys-backend/src/app/routes/index.ts:84-85

```
// EmailController.sendCustomEmail (linha 88-109)
async sendCustomEmail(req: Request, res: Response) {
  const { to, subject, body } = req.body;
  ...
  await transporter.sendMail({
    from: `${tenant.name} <${process.env.SMTP_USER}>`,
    to: Array.isArray(to) ? to : [to],
    subject,
    html: body, // <-- HTML 100% controlado pelo usuário, sem escape
  });
}

// routes/index.ts:85 - nenhum requireRole()
router.post("/api/email/send-custom", email.sendCustomEmail);
```

Por que é explorável: `body` chega diretamente do JSON do request e é usado como `html` do e-mail sem qualquer sanitização (o projeto não tem `sanitize-html`, `dompurify` ou similar em nenhum `package.json`). Além disso, a rota não exige `requireRole("admin")` nem verifica `permissions` (achado F2) — qualquer usuário autenticado do tenant, mesmo com papel "user", pode chamá-la. `to` também não é validado contra os clientes do tenant, permitindo envio para qualquer endereço externo.

Condição de explorabilidade: Requer apenas uma conta autenticada de qualquer papel no tenant (trial incluso).

Impacto: Uso da credencial SMTP corporativa como relay para phishing/HTML malicioso arbitrário enviado a qualquer destinatário, em nome da corretora.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: `/api/email/send-custom` agora exige `requireRole("admin")` e o campo `body` passa por `sanitize-html` (allowlist de tags/atributos) antes do envio.

Alta**[F5] Credencial de banco de dados com default hardcoded "postgres/postgres"****Categoria:** 4. Chaves expostas**Arquivo(s):linha(s):** graosys-backend/src/database/data-source.ts:39-41

```
host: process.env.TYPEORM_HOST || "localhost",
port: Number(process.env.TYPEORM_PORT) || 5432,
username: process.env.TYPEORM_USERNAME || "postgres",
password: process.env.TYPEORM_PASSWORD || "postgres",
database: process.env.TYPEORM_DATABASE || "graosys",
```

Por que é explorável: Caso as variáveis `TYPEORM\_\*` não sejam definidas (branch usada quando não há `DATABASE\_URL`/`POSTGRES\_URL`), a aplicação tenta conectar com usuário/senha `postgres`/`postgres` — uma credencial padrão amplamente conhecida. Não há validação de startup que rejeite essa combinação em `NODE\_ENV=production`.

Condição de explorabilidade: Explorável se o banco de destino também usar credenciais padrão (comum em containers Postgres locais/mal configurados) e estiver acessível na rede.

Impacto: Acesso não autorizado ao banco de dados completo (todos os tenants) caso a instância Postgres aceite a credencial default e esteja exposta.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: `validateEnv()` também rejeita, em produção, credenciais de banco ausentes ou iguais aos defaults conhecidos (`postgres`/`postgres`).

Média

[F6] HTML de e-mail transacional de contrato interpola campos do usuário sem escape**Categoria:** 5. Inputs sem tratamento (XSS)**Arquivo(s):linha(s):** graosys-backend/src/app/controllers/EmailController.ts:112-149

```
function buildContractHtml(contract, tenant) {
  return (role, recipientName) => `
    ...
    <p>Para <strong>${recipientName}</strong>, </p>
    ...
    <td>${contract.name_product}</td>
    <td>${contract.pickup_location || "-"}</td>
    ${contract.observation ? `<tr>...${contract.observation}</tr>` : ""}
  `;
}
```

Por que é explorável: `recipientName` (derivado de `contract.seller`/`contract.buyer`, arrays de texto livre informados na criação do contrato), `contract.observation`, `contract.name\_product`, `contract.pickup\_location` e `contract.payment` são interpolados diretamente em um template de HTML enviado por e-mail (`nodemailer`, `html: contractHtml(...)`), sem nenhuma função de escape (`encodeURIComponent`, `escape-html`, etc.). Qualquer usuário com permissão de criar contrato (todo usuário autenticado, ver F2) pode injetar HTML nesses campos.

Condição de explorabilidade: Requer que o contrato criado com payload malicioso (`<a href="javascript:...">` ou tags que alterem o layout/insiram links de phishing) seja posteriormente enviado por e-mail via POST /api/email/send-contract.

Impacto: Injeção de HTML/links em e-mails corporativos legítimos, usados para phishing contra vendedores/compradores destinatários do contrato.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: todos os campos de contrato interpolados em `buildContractHtml` (EmailController.ts) agora passam por uma função `escapeHtml()` antes de entrar no template de e-mail.

Baixa

[F7] Coluna password sem select:false — depende de disciplina manual em cada query**Categoria:** 4. Chaves expostas**Arquivo(s):linha(s):** graosys-backend/src/app/entities/User.ts:24

```
@Column()
password: string; // sem { select: false }
```

Por que é explorável: Todas as leituras de usuário auditadas (UserController.getAll/getById/getProfile, SessionController) hoje evitam vaziar o hash — por `select` explícito de colunas ou por desestruturação `const { password: \_, ...userData }`. Porém, como a coluna não tem `{ select: false }` no nível da entidade, basta um novo endpoint futuro fazer `userRepo.findOne({ where: { ... } })` sem selecionar campos explicitamente para vaziar o hash bcrypt do usuário na resposta — não há rede de proteção estrutural, só convenção.

Condição de explorabilidade: Não explorável hoje (nenhum endpoint atual vaza o campo) — é uma lacuna de defesa em profundidade.

Impacto: Risco de regressão futura: um novo endpoint pode vaziar hashes de senha por omissão.

Corrigido

Correção aplicada: Corrigido em 15/09/2026: coluna `password` marcada com `{ select: false }` (User.ts); os dois pontos que precisam do hash (login e resetPassword em SessionController.ts) agora o solicitam explicitamente via `select`.

Recomendações Priorizadas

P1

Substituir todo `Object.assign(entity, req.body)` e `{ ...req.body }` em `create/update` (`Client`, `GrainContract`, `Billing`, `Product`, `ProductTable`) por uma whitelist explícita de campos permitidos, ignorando sempre `id` e `tenant_id` do body (ver F1).

P1

Implementar de fato o middleware de checagem de `permissions` (`view/create/edit/delete` por módulo) em todas as rotas de escrita, ou remover a tela de Controle de Acesso até que a enforcement exista, para não criar falsa sensação de segurança (ver F2).

P2

Adicionar validação de startup que aborte o boot da aplicação em `NODE_ENV=production` se `JWT_SECRET` estiver ausente/curto ou igual ao valor de exemplo, e o mesmo para credenciais de banco default (ver F3, F5).

P2

Restringir `/api/email/send-custom` a `requireRole("admin")` (ou à permissão de módulo correspondente) e sanitizar `body` com uma biblioteca como `sanitize-html` antes de repassar a nodemailer (ver F4).

P2

Aplicar escape de HTML (ex.: uma função `escapeHtml()` central) em todos os campos de contrato interpolados em `buildContractHtml` antes de montar o template de e-mail (ver F6).

P3

Adicionar `{ select: false }` na coluna `password` da entidade `User` como defesa em profundidade contra vazamento futuro do hash (ver F7).

P3

Revisar a configuração de CORS (`origin: process.env.FRONTEND_URL || "*"` combinado com `credentials: true`) para nunca cair no wildcard em produção, e evitar que `errorMiddleware` repasse `err.message` cru para qualquer tipo de exceção.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de issues no GitHub. Achados triviais/relacionados foram agrupados para evitar spam de issues. **Nota:** todos os achados abaixo já foram corrigidos no código (ver "Correção aplicada" em cada achado, seção anterior); as issues são mantidas como registro/trilha de auditoria e podem ser abertas e fechadas imediatamente referenciando o commit da correção.

--- ISSUE 1 ---

Título: [Segurança] Mass assignment de ID/tenant_id permite sequestro de registros entre tenants (Client, Contract, Billing, Product, ProductTable)

Labels: security, critical

Descrição do problema

Os métodos `create` e `update` de `ClientController`, `GrainContractController`, `BillingController`, `ProductController` e `ProductTableController` constroem/atualizam a entidade a partir de `req.body` sem whitelist de campos (`{ ...req.body, tenant\_id: req.user.tenant\_id }` no create e `Object.assign(entity, req.body)` no update). O TypeORM decide INSERT vs UPDATE consultando o banco pelo valor **atual** da chave primária do objeto no momento do `save()` (`SubjectDatabaseEntityLoader`), então um `id` (ou `tenant\_id`) enviado no body é aceito e usado para localizar/gravar QUALQUER linha da tabela, de qualquer tenant – não apenas as do usuário autenticado.

Por que é explorável

A checagem `tenant\_id: req.user.tenant\_id` só é aplicada no `SELECT` inicial (que sempre localiza um registro do PRÓPRIO atacante); o `save()` final grava por chave primária, ignorando esse filtro. Basta o atacante conhecer o UUID de um registro de outro tenant (exposto em URLs da própria app, relatórios exportados, e-mails de contrato) para sobrescrevê-lo com dados arbitrários e o `tenant\_id` do atacante.

Evidência

```
```ts
// graosys-backend/src/app/controllers/ClientController.ts:46-53
async update(req: Request, res: Response) {
 const client = await clientRepo.findOne({ where: { id: req.params.id, tenant_id: req.user.tenant_id } });
 if (!client) return res.status(404).json({ error: "Cliente não encontrado" });
 Object.assign(client, req.body); // <-- aceita id/tenant_id arbitrários
 await clientRepo.save(client);
 return res.json(client);
}
```
```

Mesmo padrão em:

- `ClientController.ts:9` (create)
- `GrainContractController.ts:9-16,51` (create/update)
- `BillingController.ts:8,48` (create/update)
- `ProductController.ts:11,33,50,72` (create/update de Product e ProductTable)

Impacto

Quebra total do isolamento multi-tenant na camada de escrita: um usuário de qualquer tenant (mesmo trial) pode corromper ou sequestrar clientes, contratos, cobranças, produtos e mesas de preço de outro tenant.

Sugestão de correção

Substituir os spreads/Object.assign por uma função de whitelist explícita por entidade, que nunca aceite `id` nem `tenant\_id` vindos do body, ex.:

```
```ts
const { id, tenant_id, ...safeBody } = req.body;
Object.assign(client, safeBody);
```
```

(idealmente validado com um schema – zod já é dependência do frontend, pode ser adotado no backend também).

Critérios de aceite

- [] Nenhum handler de create/update aceita `id` ou `tenant\_id` vindos do body
- [] Whitelist explícita de campos aplicada em Client, GrainContract, Billing, Product e ProductTable (create e update)
- [] Teste automatizado que tenta sobrescrever um registro de outro tenant via `id`/`tenant\_id` no body e espera 404/erro
- [] Revisão de regressão nas 5 entidades listadas

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Permissões granulares por módulo (view/create/edit/delete) não são aplicadas em nenhum endpoint

Labels: security, critical

Descrição do problema

A tela `/admin/access` permite configurar, por usuário não-admin, permissões granulares (`view`/`create`/`edit`/`delete`) por módulo (`contracts`, `clients`, `execution`, `billing`, `reports`), persistidas em `User.permissions`. Nenhum controller do backend (nem componente do frontend fora da própria tela de configuração) lê ou valida esse campo antes de autorizar uma ação.

Por que é explorável

Basta autenticar como qualquer usuário não-admin. As únicas verificações de autorização existentes no backend são `authMiddleware` (JWT válido) e `requireRole("admin")` (aplicado apenas em rotas administrativas e nos deletes). Create/edit de contratos, clientes, cobranças e relatórios ficam liberados independentemente da configuração de `permissions`.

Evidência

```ts

// graosys-backend/src/app/routes/index.ts:51-73 (trecho)

```
router.post("/api/clients", client.create); // sem check de permissão
router.patch("/api/clients/:id", client.update); // sem check de permissão
router.post("/api/contracts", contract.create); // sem check de permissão
router.post("/api/billings", billing.create); // sem check de permissão
```
```

`permissions` só aparece em `UserController.ts` (armazenar/retornar) e em `SessionController.ts` (repassar no token) – nunca em uma checagem de autorização.

Impacto

Falsa sensação de segurança para administradores que configuram segregação de funções; qualquer usuário mantém acesso de escrita total aos módulos operacionais.

Sugestão de correção

Criar um middleware `requirePermission(module, action)` que leia `req.user.permissions[module]` e aplicá-lo em todas as rotas de create/update/delete de contracts, clients, billing e reports; ou, como medida imediata, remover/ocultar a tela de Controle de Acesso até a enforcement existir.

Critérios de aceite

- [] Middleware de checagem de permissão implementado e coberto por teste
- [] Aplicado em todas as rotas de escrita dos módulos contracts/clients/billing/reports
- [] Usuário sem permissão de `create`/`edit` em um módulo recebe 403 ao tentar a ação via API diretamente

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Defaults inseguros hardcoded (JWT_SECRET e credencial de banco) sem validação de startup

Labels: security, high

Descrição do problema

Dois segredos críticos têm fallback hardcoded no código-fonte, usados silenciosamente quando a variável de ambiente correspondente não está definida:

- `JWT\_SECRET` → fallback `"secret"`
- Credenciais Postgres → fallback `postgres`/`postgres`

Não existe nenhuma validação de startup que rejeite esses defaults quando `NODE\_ENV=production`.

Por que é explorável

Se a variável de ambiente correspondente faltar no ambiente de deploy (erro de configuração), a aplicação sobe normalmente usando o segredo público – permitindo forjar JWTs válidos (incluindo `role: admin`) ou acessar o banco com credencial padrão, sem qualquer alerta.

```
## Evidência
```ts
// graosys-backend/src/app/middlewares/authMiddleware.ts:32
const decoded = jwt.verify(token, process.env.JWT_SECRET || "secret") as TokenPayload;

// graosys-backend/src/app/controllers/SessionController.ts:45
process.env.JWT_SECRET || "secret",

// graosys-backend/src/database/data-source.ts:39-41
username: process.env.TYPEORM_USERNAME || "postgres",
password: process.env.TYPEORM_PASSWORD || "postgres",
database: process.env.TYPEORM_DATABASE || "graosys",
```

## Impacto
Personificação total de qualquer usuário/tenant (via JWT forjado) ou acesso irrestrito ao banco de dados, caso o ambiente de produção não sobrescreva os defaults corretamente.

## Sugestão de correção
No bootstrap (`server.ts` / `api/index.ts`), validar em `NODE_ENV=production`:
```ts
if (process.env.NODE_ENV === "production") {
 if (!process.env.JWT_SECRET || process.env.JWT_SECRET.length < 32) {
 throw new Error("JWT_SECRET ausente ou fraco em produção");
 }
 if (!process.env.TYPEORM_PASSWORD && !process.env.DATABASE_URL) {
 throw new Error("Credencial de banco não configurada em produção");
 }
}
```

## Critérios de aceite
- [ ] Aplicação recusa subir em produção sem `JWT_SECRET` forte configurado
- [ ] Aplicação recusa subir em produção sem credencial de banco explícita
- [ ] Fallbacks hardcoded removidos ou mantidos apenas para desenvolvimento local, de forma explícita (ex.: checando `NODE_ENV !== "production"`)

```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] HTML sem sanitização em e-mails (send-custom sem gate de papel + template de contrato sem escape)

Labels: security, high

Descrição do problema

1. `POST /api/email/send-custom` aceita `body` como HTML livre e o envia via `nodemailer` sem qualquer sanitização, para qualquer destinatário, sem exigir papel administrativo.
2. `buildContractHtml` (template de e-mail de contrato) interpola campos de texto livre do contrato (`observation`, `name\_product`, `pickup\_location`, `payment`, nomes de vendedor/comprador) diretamente na string de HTML, sem escape.

Por que é explorável

Não há biblioteca de sanitização (`sanitize-html`, `dompurify`, etc.) em nenhum `package.json` do projeto. Qualquer usuário autenticado (independente de papel – ver issue #2) pode: (a) chamar `send-custom` diretamente com HTML malicioso para qualquer endereço, usando a credencial SMTP da empresa; ou (b) cadastrar um contrato com HTML malicioso em `observation`/`pickup\_location`/etc. e disparar o envio via `send-contract`.

Evidência

```
```ts
// graosys-backend/src/app/controllers/EmailController.ts:88-109
async sendCustomEmail(req: Request, res: Response) {
 const { to, subject, body } = req.body;
 ...
 await transporter.sendMail({ ..., html: body }); // sem sanitização
}
```

```

```
// routes/index.ts:85 – sem requireRole()
router.post("/api/email/send-custom", email.sendCustomEmail);
```

```
// EmailController.ts:126-134 (trecho do template)
<td>${contract.name_product}</td>
<td>${contract.pickup_location || "-"}</td>
${contract.observation ? `<tr>...${contract.observation}</tr>` : ""}
```
```

#### ## Impacto

Relay de phishing/HTML malicioso usando a identidade e a credencial SMTP da corretora; injeção de conteúdo em e-mails transacionais legítimos enviados a clientes externos (vendedores/compradores dos contratos).

#### ## Sugestão de correção

- Restringir `/api/email/send-custom` a `requireRole("admin")` (ou à permissão do módulo, uma vez implementada na issue #2) e sanitizar `body` com `sanitize-html` antes de repassar ao `nodemailer`.
- Criar uma função `escapeHtml()` central e aplicá-la em todos os campos de contrato interpolados em `buildContractHtml`.

#### ## Critérios de aceite

- [ ] `send-custom` exige papel admin (ou permissão equivalente)
- [ ] `body` de `send-custom` passa por sanitização antes do envio
- [ ] Todos os campos de contrato em `buildContractHtml` são escapados
- [ ] Teste com payload `

### --- FIM ISSUE 4 ---

### --- ISSUE 5 ---

**Título:** [Segurança] Hardening diversos: coluna password sem select:false, CORS wildcard+credentials, vazamento de err.message

**Labels:** security, low

#### ## Descrição do problema

Três itens de baixo risco, agrupados por serem melhorias de defesa em profundidade e não vulnerabilidades exploráveis diretamente hoje:

1. `User.password` não tem `{ select: false }` – depende de disciplina manual em cada query para não vazarem o hash bcrypt.
2. CORS configurado com `origin: process.env.FRONTEND\_URL || ""` e `credentials: true` – se a env var não for definida, qualquer origem é aceita.
3. `errorMiddleware` retorna `err.message` cru como resposta 400 para qualquer exceção, podendo vazarem detalhes internos (ex.: mensagens de constraint do banco).

#### ## Evidência

```
``ts
// graosys-backend/src/app/entities/User.ts:24
@Column()
password: string; // sem select: false

// graosys-backend/src/app.ts:10-13
app.use(cors({ origin: process.env.FRONTEND_URL || "", credentials: true }));

// graosys-backend/src/app/middlewares/errorMiddleware.ts:5-7
if (err.message) return res.status(400).json({ error: err.message });
```
```

Impacto

Baixo isoladamente; aumentam a superfície de risco caso outras proteções falhem (regressão futura, erro de configuração de deploy, exceções inesperadas).

Sugestão de correção

- Adicionar `{ select: false }` na coluna `password`.
- Nunca deixar `FRONTEND\_URL` cair no wildcard em produção; validar no startup.
- Retornar apenas mensagens de erro conhecidas/tratadas (whitelist), logando o detalhe completo só no servidor.

Critérios de aceite

- [] `password` marcado com `select: false` e endpoints existentes continuam funcionando (selects explícitos ainda funcionam)
- [] CORS falha o startup em produção se `FRONTEND\_URL` não estiver definida
- [] `errorMiddleware` não repassa mensagens de exceções não tratadas ao cliente

--- FIM ISSUE 5 ---
